

Deterministic AI Development: A Workflow Overview

Akien Maciain, Test Automation Architect

Introduction

Models misbehave most when given the most latitude. In that regard, skills are wonderful tools. Skills are like checklists of things to do. They keep the model very focused on doing just what you want.

Skills by themselves do not complete the work though. Skills work best when part of an overall workflow that handles design and implementation.

My own experience is with building [akienm/TheIgor](#). An ongoing research project into using graph trees for reasoning. ~200K lines of code, which works, and has only ever been seen by Claude. Model routing is now codified by task type: Opus for design audits and the expert panel, Sonnet for sprint execution, day audits, and code-quality review, Haiku for mechanical work (ticket filing checks, debris cleanup, snapshot tooling, commit). I have reviewed only a dozen or so lines of the code myself.

Every one of the things below came out of some pain point. Often centered around keeping Claude up to date in some way.

This describes my workflow and more importantly the rationale for each piece. It's divided into:

TRACKING – What we track

WORKFLOW – What my workflow is

SKILLS – A brief note on audits and then a complete skills list by alpha

I tend to have one really long running Claude Code chat window for the work. The mechanisms below allow me to take advantage of token caching by electing what order I want to tackle things in. External Claude Code shells (minions) do most of the work.

This allows me to treat Claude as a savant. Brilliant in some ways, but forgetful in others. And not fight that, but build around it – a resilient structure that keeps its forgettings from becoming an issue. I still have to keep a lot in my head. But it takes care of the rest of the work.

Tracking

We track goals, ideas, decisions, tickets, slates, occasionally epics, architecture, the code, and the memory palace. (Memory Palace is a memory tool that helps students study, and Claude/Igor to find things quickly and with fewest tokens.)

Goal: It's just whatever we want to make functional. For a while, the goal was optimizing database usage. At the moment, it's 'Igor can pick up a ticket and complete it'. Goals are not tracked anywhere formally, but they are a direction for whatever the current work is.

Ideas: It's just a folder. One text file per idea. These are pasted in and discussed.

Why: Lets me brainstorm and chat with free AIs in web chats to sort details with no paid tokens. This isn't an implementation, it's an idea that will become a discussion.

Decisions: /design starts a design session.

This can be a pretty free form discussion. Decisions are where a point of design, a subsystem, or set of related tickets have their details worked out to a level of clarity and determinism that Haiku can reliably build it without fail.

Why: Because by doing the coding in Haiku (or Sonnet for medium-complexity work), I save tokens. By doing a detailed enough design, the smaller model can do it without needing the big brains of its big siblings.

One idea at a time can be pasted in, or they can be done in (usually) related groups.

Sometimes all we're deciding on is which open tickets to tackle next and in what order. This often looks like:

"Of the open tickets, and using greedy ticket selection, which tickets remain open that are relevant to the goal and in what order should we tackle them?"

And then it's either I have questions or input, or /decided.

The last questions I ask at the end of the design step:

- What am I missing?
- What could we do better?

Why: Because for all my experience, there's still plenty I don't know. These two questions have turned up all kinds of new things. They're amazing.

When it's done, /decided.

This creates an entry in the decisions log. Each decision also creates one or more tickets. As each ticket is created, `/audit-ticket` runs the filing-time audit:

- Is this a dupe with any other tickets?
- Is this already done in the code?
- Is this blocked by anything else that's pending?
- Is this well representative of the likely size of the work?
- Scope creep: should this be broken up?
- What is the passing condition?
- Which files will this touch?
- Any high-inertia files? (files whose changes might be high risk)
- Does the description match the title?
- What documentation will be updated?
- Do the design rules apply?
 - no-sqlite
 - oop-first
 - docs-in-code
 - no schema changes
 - all try/excepts will at minimum log the occurrence
 - names for variables and methods all describe what is being done
- What tests will we build/run to prove this works?
- Rollback plan for high-inertia file touches

Notes link the ticket back to its decision, and the decision is updated with its tickets.

Slate:

A slate is a day's work. Slates contain:

- In-flight – what are we doing right now
- Planned – what's still planned for this slate
- Ad hoc – reactive additions
- Done today

When the day's slate is done, we run `/day-close`.

Epics: are just groups of related tickets. Like 'we're working on cognition today!'

Architecture & Code: Everything is documented at a high level in the project in MD files in the repo. The AI will read the architecture files to sort which files have to be modified. The actual documentation for each file will be IN THE FILE. At the top. So the AI can read it first. But all the key points like how things work – that's in the project docs. Any ticket that updates the architecture has to update that tree of files.

Why: Saves tokens. Key points are read in root MD files, then the code files are read at the top, then the functional code itself. Minimizes the number of files that have to be looked at.

Memory Palace: A structured Postgres tree (`theigors/rules/*`, `theigors/decisions/*`, `theigors/audits/*`, `theigors/infrastructure/*`, etc.) that serves as the canonical index for conventions, rules, audit telemetry, and decision history. `CLAUDE.md` is a thin bootstrap shim; the palace is the source of truth.

Why: Palace nodes can be read individually (`memory_get`), searched (`memory_search`), or bulk-loaded by type. The model spends zero tokens re-deriving conventions it already decided.

Preferred Paths: A palace subtree (`theigors/rules/preferred_paths/*`) cataloging deprecated patterns alongside their preferred replacements – e.g. raw `psql` calls vs. the MCP proxy, `print()` vs. the IgorBase logger, direct DB writes vs. `cortex.store()`. A scan tool watches 60 days of git history for regressions and surfaces candidates for review, never auto-filing.

Workflow

My workflow mostly falls out like this:

`/context-load` – the agent reads the project overview, the list of available tools, palace rules, and today's slate. The agent gets centered. About 2K tokens.

`/design` – we work our way through any tickets that are blocked.

`/note` – adds any random note that might be important later in the day's slate.

`/ticket` – any issues that come up along the way so we address them later.

`/decided` – all issues under discussion are now resolved enough to go to sprint. Launches the `/audit-design` skill to validate the design before tickets are written. Ticket anything we've been talking about that isn't ticketed, run `/audit-ticket` on each draft, and get it ready for sprinting.

`/sprint` `/sprint-batch` – sprint a ticket or sprint a large batch of tickets. Sprint-batch calls sprint over and over. `/sprint` also calls `/savestateauto` on completion.

`/audit-encode` – runs automatically between plan approval and the first edit in `/sprint`. Validates that named files exist, symbols exist, preferred-paths rules are satisfied, and the test plan is named. Haiku-speed. Escalates to Sonnet on high-inertia touches.

`/fixit` – shorthand for `/decided` + `/sprint-batch` in one go.

`/day-close` – cleans up, runs `/audit-day`, closes out the day's slate.

Skills

Audit – The Pyramid

We have a family of scoped audits, each targeting a different failure class and running at the cheapest model that can reliably catch it.

Pre-filing (`/audit-ticket`, Haiku): runs on every ticket draft before it lands in the queue. Duplicate detection, already-done-in-code check, HIGH-inertia pre-approval gate, scope-creep split, build-tightness grade, design-rule checks (palace-loaded at filing time). See Tickets section above.

Pre-code (`/audit-encode`, Haiku → Sonnet): runs between plan approval and first edit. File/symbol existence, HIGH-inertia reaffirmation, preferred-paths compliance, test plan named, docstring plan, diff-size vs ticket-size estimate.

Post-code (`/audit-smell`, Sonnet): runs after code is written, before tests. Checks for premature abstractions, bespoke logic where a standard pattern exists, missing log calls, misleading names, over-complex conditionals, and test shape adequacy.

Post-build debris (`/audit-debris`, Haiku): cleanup pass after commit. Temp/artifact files, debug prints/breakpoints, log-size growth, test DB cleanup (live rows in test schemas), file placement, docstring rot on touched load-bearing files, subsystem index drift, commented-out code.

Daily cross-session (`/audit-day`, Sonnet): run by `/day-close`. Inherits all 18 day-close-audit static checks plus: fix-one-leave-many sweep (function signature changed in N callers but M others missed), watch-for notes from prior runs (hit/age/expire), subsystem index vs. reality, inertia tag drift, TWM coverage gaps, habit health. Auto-drafts a scan-for-rest ticket to `/tmp/` when fix-one-leave-many is detected.

Design-gate (`/audit-design`, Opus): runs at the opening of a `/decided` block. Reviews the design itself: inertia levels, blast radius, reversibility, test strategy, simplicity vs. bespoke complexity.

Expert panel (`/audit-expert`, Opus): broadest-lens review. Each expert sees the whole codebase through their field's sharpest questions – not “is this code clean?” but “is this system doing what this discipline demands?” Per expert: ≤5 severity-tagged observations, ≤2 watch-for notes (stored in palace with TTL ≤ 14 days), 0-1 candidate ticket drafts routed through `/audit-ticket` before filing.

#	Expert	Broadest lens
1	Cognitive Scientist	Is reasoning architecture consistent with human cognition models?
2	Systems Architect	Is subsystem decomposition clean? Coupling, cohesion, blast radius.
3	Safety Engineer	What are the failure modes? Runaway processes, unrecoverable states.
4	HCI Specialist	Is Igor legible to its users? Feedback quality, error clarity, trust signals.
5	Distributed Systems	Is the multi-instance design sound? Consistency, idempotency, clock drift.
6	ML Engineer	Is the learning architecture coherent? Feedback loops, distribution shift.
7	Process / Meta Engineer	Is the development process self-improving? Audit ROI, tech-debt rate.
8	Security Engineer	What can go wrong from adversarial inputs? Injection, secret exposure.
9	Reliability Engineer	What does the on-call story look like? MTTR, alerting gaps.
10	Data Engineer	Is the persistence layer sound? Schema drift, migration safety, lineage.
11	Product Manager	Is Igor making progress toward its stated goal? Velocity, blocker patterns.

Cadence: weekly runs 3 random experts; monthly runs the full panel (with Ultraview on HIGH findings).

Meta-audit (`/audit-audits`, Sonnet/Opus): audits the audit pyramid itself – watch-for TTL compliance, telemetry sampling uniformity, check confidence calibration, cadence adherence, findings-to-ticket conversion rate. Runs monthly or on demand.

All audit levels emit structured telemetry to the palace (`theigors/audits/<level>/runs/<timestamp>`). This creates a uniform time-series for trend analysis – findings per week, checks fired vs. amended vs. discarded, watch-for hit rates.

Full Skills List (alpha)

`/audit-audits` – meta-audit over all audit telemetry; checks cadence, TTL compliance, confidence calibration

`/audit-day` – cross-day code health: inherits all day-close-audit checks + fix-one-leave-many sweep + watch-for management + telemetry

`/audit-debris` – post-commit debris cleanup: temp files, debug artifacts, docstring rot, test DB cleanup, file placement

`/audit-design` – design-gate review before `/decided`: inertia, blast radius, reversibility, simplicity

`/audit-expert` – 11-expert broadest-lens panel; weekly (3 random), monthly (full), on-demand by area

`/audit-precode` – pre-edit plan validation: file/symbol existence, preferred-paths, HIGH-inertia reaffirmation, test plan named

`/audit-smell` – post-code quality scan: premature abstractions, missing log calls, misleading names, bespoke vs. standard patterns

`/audit-ticket` – filing-time ticket audit (replaces `/review Mode A`): duplicate, already-done, scope, HIGH-inertia gate, design-rules, build-tightness grade

`/commit` – does the commit, pull (and merge), push

`/context-load` – loads project overview, palace rules, today's slate, recent decisions, pending approvals, inbox

`/day-close` – closes out the day: `savestateauto`, slate finalization, `/audit-day`, docs commit, GitHub Discussion, push

`/day-close-audit` – static 18-step debris and hygiene check (tests, file placement, smells, registry, inertia, threads, logs, burn rate, schema, dead code, duplication, habit health, TWM coverage, dependencies, credentials, simplification, registered checks, wiring, capability-map drift)

`/decided` – closes a design block → batch tickets via `/audit-ticket`, writes decision to palace and log, appends to slate

`/deep-audit` – legacy alias (superseded by `/audit-expert` + `/audit-day` pyramid)

`/design` – design-mode session marker; writes `DESIGN_START` to slate, sets `design_mode` flag

`/export-chat` – exports current Claude Code chat window to a dated markdown file (works around tmux scrollbar limits)

`/fixit` – shorthand for `/decided` + `/sprint-batch` on the just-filed tickets in one go

`/map-igor` – on-demand JSON snapshot of Igor's full state (15 sections: palace tree, tickets, gates, processes, schema, logs, inbox, channels, etc.); Haiku; 14-day TTL

`/note` – adds a random note to the day's slate

`/readigor` – reads the web interface for Igor so Claude can be brought up to speed without pasting

`/readinbox` – reads Igor's inbox (messages from build processes, Claude, internal subsystems)

`/review` – standalone code/plan/PR review (Mode B); filing-time ticket audit now lives in `/audit-ticket`

`/savestate` – full session close: `savestateauto` + compose preserve string + inject compaction

`/savestateauto` – lightweight state flush called by other skills: writes in-flight hypothesis to slate, clears debug flag, emits compact preserve string

`/sprint` – per-ticket execution loop: claim → audit-precode → infrastructure brief → pull+work → cleanup → test → commit+push → close → `savestateauto`

`/sprint-batch` – multi-ticket sprint: topo-sort by dependencies, shared setup once, per-ticket loop, batch teardown

`/test-fix` – test/fix/test-again loop for failing suites

`/ticket` – creates or updates a ticket; runs `/audit-ticket` on each draft before filing

`/validate-files` – file placement audit (absorbed into `/audit-debris`; still invocable standalone)

Copilot's Review

Objective Assessment of the Workflow (Fact-Based)

Primary design principle: Reduce model variance by minimizing freedom at execution time and concentrating ambiguity in early, cheap contexts (ideas/design).

Key control mechanism: Explicit separation of ideas → decisions → tickets → execution, with a hard determinism boundary at `/decided`.

Cost efficiency: - Expensive reasoning is done once in design. - Implementation is delegated to smaller models with constrained skills. - Context size is aggressively reduced via `/savestate` and compaction.

State management: - All durable state (goals, decisions, tickets, slates, architecture) is externalized. - Model context is treated as disposable compute, not memory.

Documentation strategy: - Architecture exists in repo-level Markdown. - File-level intent is documented at top of each file. - Docs are timestamped and compared against commit times to detect drift. - Daily cleanup maintains consistency and correctness.

Execution reliability: - Tickets are audited before execution for scope, duplication, risk, and testability. - /sprint enforces a fixed execution sequence (plan → implement → test → commit → save).

Quality control: - Multiple audits exist, each targeting a different failure class at the cheapest model tier. - Expert audits run specialists in parallel, incurring a single token batch, invoked selectively.

Outcome: Predictable execution. Low rework. Reduced token consumption. Stable progress across long-running work.

ChatGPT's Review

Technical Workflow: Deterministic AI Orchestration

Objective: To minimize LLM "drift" and token consumption by establishing a rigid separation between Design (intent) and Implementation (execution).

1. Core Philosophy: The Latitude Principle

Models misbehave most when given the highest degree of latitude. To ensure production-grade output, we utilize Skills (checklists) and Workflows to constrain the AI's focus to a specific, deterministic path.

2. Information Architecture (Tracking)

We maintain a strict hierarchy of data to ensure the AI remains centered without excessive context-loading.

- *Ideas (Pre-Production):* Free-form brainstorming occurs in isolated environments to refine concepts before they enter the billable token stream.
- *Decisions (/design):* A formal session where subsystems are refined to a level of clarity and determinism that allows smaller, faster models to execute the build without ambiguity.
- *Tickets & Audits:* Every decision generates audited tickets. Before work begins, we validate scope, context conflicts, risk, and testability.
- *Slates:* A "Slate" represents a single day's work-unit, providing a persistent record of "In-Flight" and "Completed" tasks, independent of the AI's ephemeral memory.

3. Operational Workflow

The lifecycle follows a standard command-driven pattern to maintain state: /context-load → /design → /decided → /sprint → /day-close.

4. Multi-Layered Audit Strategy

Changes pass through a layered review process keyed to failure class and model tier:

- *Pre-filing (/audit-ticket, Haiku):* Validates requirements, duplicates, scope, and design rules before the ticket lands in the queue.
- *Pre-code (/audit-precode, Haiku):* Confirms files and symbols exist, preferred patterns are used, test plan is named, before the first edit.
- *Post-code (/audit-smell, Sonnet):* Code quality scan – abstractions, naming, log coverage – before tests run.
- *Post-commit cleanup (/audit-debris, Haiku):* Removes artifacts, debug prints, stale docstrings.
- *Daily cross-session (/audit-day, Sonnet):* 18-step debris check plus fix-one-leave-many sweep, watch-for management, inertia drift detection.
- *Expert Panel (/audit-expert, Opus):* 11-discipline review; weekly (3 experts), monthly (full panel). Each expert emits severity-tagged observations, watch-for notes with TTL, and candidate ticket drafts.
- *Meta-audit (/audit-audits):* Audits the audit pyramid itself for cadence, calibration, and coverage.

5. Architectural Efficiency

- *Externalized State:* By storing state in palace DB / markdown files, we avoid "context bloat."
- *Docs-in-Code:* High-level architecture is maintained in the repo; functional documentation lives at the top of each file.
- *Memory Palace:* Canonical rules, decisions, and infrastructure briefs live in a queryable Postgres tree. The model reads only what's needed for the current ticket.

Summary: This workflow moves the AI from a "Creative Assistant" role to a "Deterministic Executor," resulting in higher code reliability, lower operational costs, and a clear audit trail for all system changes.

For Additional Background

- [GitHub – akienm/agent_datacenter](#) – runtime substrate: IMAP bus, device model, rack, installer
- [GitHub – akienm/TheIgor](#) – Igor: graph-matrix reasoning engine, persistent Postgres memory, habit scoring
- [TheIgor – working_with_claude.md](#)